

VEHICLE AUCTION PLATFORM

Complete Study & Viva Reference

Kavindu Kamil Jehan Hamid

Index: R222226 | IT5106 Software Development Project

Client: Tapro Japan Co. Ltd. | April 2026

How to Use This Document

This document covers everything about your project from first principles — client discussion through architecture, all bugs found and fixed, every design decision made, and deployment.

Study it section by section. The "Bugs & Fixes" chapter documents every real problem discovered during development with the exact reasoning behind each fix — this is likely what the viva panel will probe.

The "Code Change Challenge Guide" chapter at the end is specifically designed for your exam scenario.

1. Client Context & Problem Domain

1.1 Who is the Client

Tapro Japan Co. Ltd. is a vehicle exporter based in Tokyo, Japan. The business acquires used vehicles from Japanese auto auctions held weekly and resells them to international buyers in markets such as Sri Lanka, Kenya, and the Middle East. The client is a small private operation with two business owners.

1.2 The Existing System (Before This Project)

Before this platform existed, the entire auction workflow ran through informal messaging channels. Understanding this is critical because every feature in the system exists to solve one of these specific pain points.

Phase	How it worked (old way)
Vehicle Listing	Photos taken manually, specs recorded on paper or in notes, distributed via WhatsApp broadcast lists and Facebook posts. No consistent structure.
Bid Collection	Buyers reply to message threads. Client manually reads all replies and identifies highest bid. No real-time visibility, no enforcement of minimum increments or time limits.
Winner Selection	Client privately messages the winning bidder. Process is opaque to other participants and perceived as potentially unfair.

1.3 Identified Problems

- No audit trail — bids placed informally, impossible to track accurately
- No access control — anyone could bid, no commitment verification
- Inconsistent communication — vehicle info scattered across different threads
- No centralized history — no records of auctions, bids, or buyer activity
- No scalability — manual process becomes bottleneck as business grows
- Disagreements about winning bid — no authoritative record

1.4 What the System Must Do (In Plain English)

The platform replaces WhatsApp chaos with a structured digital workflow. Admins (the business owners) control everything. Buyers register and wait for admin approval before they can bid. The system automatically manages auction timing. Results are transparent and permanent.

2. Requirements Analysis

2.1 User Roles

Role	How Created	What They Can Do
admin	Seeded via script (never self-registered)	Full CRUD on vehicles, auctions, users. Verify/reject buyers. Close auctions. Assign winners.
buyer	Self-registers via /api/auth/register	Browse vehicles and auctions. Place bids — only after admin verification. View their own won auctions.
guest (unauthenticated)	No login required	Browse auctions and vehicles. View bid history. Cannot bid.

Critical Design Decision — No Seller Role

Early designs included a "seller" role distinct from admin. This was removed because in this business the owner IS the seller — it is a 1:N relationship (one owner sells to many buyers). Keeping a separate seller role added complexity with zero business value.

The seller_id column in the vehicles table was kept (it always refers to the admin who created the vehicle), but the seller role was eliminated from registration, route guards, and service logic.

2.2 Functional Requirements

ID	Requirement	Priority	Category
FR-01	User registration with email, password, and contact info	High	Users
FR-02	Admin verification or rejection of registered user accounts	High	Users
FR-03	Secure login with JWT-based authentication	High	Users
FR-04	Role-based access control: public, verified buyer, admin	High	Users
FR-05	Admin CRUD operations for vehicle listings (images, specs)	High	Vehicles
FR-07	Public vehicle browsing with search and filter (make, model, year, price)	High	Vehicles
FR-08	Creation of auctions with configurable start/end datetime	High	Auctions
FR-09	Automatic auction status transitions (draft→active→ended)	High	Auctions
FR-10	Verified users may place bids on active auctions	High	Bidding
FR-11	Bid validation: minimum increment, auction timing check	High	Bidding
FR-12	Real-time display of current highest bid	High	Bidding
FR-14	Complete bid history per auction	High	Bidding
FR-15	Admin dashboard: pending verifications, active auctions	High	Admin
FR-16	Admin selection of winning bidder per auction	High	Admin

2.3 Non-Functional Requirements

- Security: Passwords bcrypt-hashed (rounds ≥ 10). JWTs validate all protected endpoints. Parameterized queries prevent SQL injection.
- Performance: API response < 500 ms under normal load. Supports ≥ 100 concurrent users.
- Reliability: 99% uptime during auction periods. DB transactions for concurrent bid consistency.
- Usability: Responsive design for desktop and mobile. No training required for returning buyers.
- Scalability: Stateless backend deployable behind load balancer. PostgreSQL connection pooling via pg.Pool.

3. System Architecture

3.1 High-Level Overview

The system is a three-tier web application. Each tier has a single clear responsibility and communicates with adjacent tiers only through well-defined interfaces.

Tier	Technology	Responsibility
Presentation	React 19 + Vite + Tailwind CSS v4	Single Page Application. Renders UI, manages local state, communicates with backend via HTTP only.
Application	Node.js + Express	REST API. Enforces authentication, authorization, business rules. No business logic in frontend.
Data	PostgreSQL	Relational storage. ACID transactions for bid integrity. Connection pooling via pg.Pool.

3.2 Backend Layered Architecture

The backend follows strict layer separation. Each layer has one job and never skips layers.

```
Request → Route → Controller → Service → Repository → Database
                        ↑
                Middleware (auth, validation, error handling)
```

Layer	Responsibility
Routes	URL patterns + middleware chain only. Zero business logic. Maps HTTP verbs to controller functions.
Controllers	Thin. Call exactly one service method, send response or call next(err). No logic.
Services	All business logic, authorization checks, cross-entity rules. This is where the work happens.
Repositories	SQL queries only. No business logic. Returns raw DB rows.
Middleware	JWT auth, role enforcement, Joi validation, 404 handler, global error handler.

3.3 Why This Architecture?

- **Testability:** Services can be unit-tested without the HTTP layer. Repositories can be mocked.
- **Maintainability:** If you need to change how vehicles are listed, you only touch `vehicleRepository.js`. The change cannot break auth.
- **Security:** Authorization lives in services, not controllers, so it is always enforced even if called from tests or other services.
- **Separation of concerns:** A database schema change (adding a column) requires editing one file: the repository. Nothing else knows about column names.

3.4 Frontend Architecture

main.jsx → BrowserRouter → AuthProvider → App.jsx → AppRouter → Pages

- AuthContext: JWT stored in localStorage. Decoded client-side with atob() — no /me call on every page refresh. Provides user, token, login(), logout(), isAdmin, isVerified to all components.
- API Layer: One Axios instance (client.js) with a request interceptor that attaches the JWT. One file per resource (auth.js, auctions.js, vehicles.js, bids.js, users.js). NO Axios calls directly in components — all HTTP goes through src/api/.
- Pages: Each page owns its own data fetching + loading/error state using useState and useEffect.
- Router: React Router v7. Protected routes use ProtectedRoute component that checks JWT presence and role.

4. Database Design

4.1 Final Schema

The schema evolved through 8 incremental migrations. Understanding why it evolved is more important than memorizing the final state.

users table

Column	Type	Notes
id	SERIAL PK	Auto-incrementing primary key
email	VARCHAR(255) UNIQUE	Case-sensitive. Unique constraint prevents duplicate accounts.
password_hash	TEXT	bcrypt hash. NEVER stored as plain text. NEVER returned in API responses.
name	VARCHAR(255)	
role	VARCHAR(20)	CHECK constraint: 'admin' or 'buyer' only.
verification_status	VARCHAR(20)	CHECK: 'pending', 'verified', or 'rejected'. Replaces the boolean is_verified column (see Section 5).
created_at	TIMESTAMPTZ	Timezone-aware timestamp. Always store UTC.

vehicles table

Column	Type	Notes
id	SERIAL PK	
seller_id	INTEGER FK → users	Always the admin who created the listing.
title, make, model	VARCHAR	
year	SMALLINT	
starting_price	NUMERIC(12,2)	Decimal precision for currency. Never use FLOAT for money.
status	VARCHAR(20)	CHECK: 'draft', 'listed', 'sold'
chassis_number	VARCHAR(100) UNIQUE	NULLABLE. PostgreSQL UNIQUE allows multiple NULLs — vehicles without chassis number do not conflict.
mileage, grade	INTEGER / VARCHAR	Optional. Added in migration 004.
images	TEXT[]	PostgreSQL array of Cloudinary URLs. Added in migration 004.

auctions table

Column	Type	Notes
id	SERIAL PK	
vehicle_id	INTEGER FK → vehicles CASCADE	Deleting a vehicle deletes the auction.
title, description	VARCHAR / TEXT	
status	VARCHAR(20)	CHECK: 'draft', 'active', 'ended'
starts_at, ends_at	TIMESTAMPTZ	Used by both cron scheduler and on-read correction for status transitions.
min_increment	NUMERIC(12,2)	Minimum bid raise. 0 = just needs to beat current highest. Added in migration 006.
reserve_price	NUMERIC(12,2) NULLABLE	Hidden from buyers. NULL = no reserve. Added in migration 007.
winning_bid_id	INTEGER FK → bids	Set when admin closes or selects winner. Added in migration 003.

bids table

Column	Type	Notes
id	SERIAL PK	
auction_id	INTEGER FK → auctions CASCADE	CRITICAL — bids are scoped to an auction, not a vehicle. (Originally was vehicle_id — see Section 5 Bug #5).
user_id	INTEGER FK → users	
amount	NUMERIC(12,2)	
created_at	TIMESTAMPTZ	

4.2 Migration Strategy

Each schema change is an incremental SQL migration file. Migrations are applied in order. This is the professional standard for managing database changes over time — you never modify the schema by hand in production.

Migration File	What it Does
001_initial_schema.sql	Creates users, vehicles, auctions, bids tables. Original bids table had vehicle_id.
002_bids_auction_id.sql	Drops vehicle_id from bids, adds auction_id FK. Adds performance indexes.
003_auction_winner.sql	Adds winning_bid_id FK to auctions.
004_vehicle_fields.sql	Adds chassis_number, mileage, grade, images[] to vehicles.
005_user_verification_status.sql	Drops is_verified boolean. Adds verification_status enum. Migrates existing data.
006_auction_min_increment.sql	Adds min_increment to auctions.

Migration File	What it Does
007_auction_reserve_price.sql	Adds reserve_price (nullable) to auctions.
008_vehicle_chassis_unique.sql	Adds UNIQUE constraint to chassis_number.

4.3 Key Database Design Decisions

Why NUMERIC(12,2) for money, never FLOAT

Floating point numbers (FLOAT, DOUBLE) use binary representation and cannot represent decimal fractions exactly. For example, $0.1 + 0.2$ in floating point does not equal exactly 0.3. For financial amounts this is unacceptable. NUMERIC(12,2) stores exact decimal values.

Why highest_bid is a subquery, not a column

The initial design stored current_highest_bid as a mutable column updated on every bid insertion. This denormalized approach risks data inconsistency: if the bid INSERT succeeds but the subsequent UPDATE to current_highest_bid fails, the data is wrong. In the revised schema, highest_bid is computed at query time using a correlated subquery: `(SELECT MAX(amount) FROM bids WHERE auction_id = a.id)`. This is always accurate because it reads live data.

Why verification_status is a VARCHAR CHECK, not a lookup table

The original ERD had a separate AuctionStatus table and Role table with their own PKs, following classic normalization. However, these are fixed, non-dynamic sets of values — they will never be added to or removed at runtime. Using inline CHECK constraints eliminates unnecessary JOIN operations on every authentication and auction status query without sacrificing data integrity.

5. Bugs Discovered and Fixed — Complete Record

This is the most important chapter for viva preparation. Every issue here was discovered through systematic code review against requirements. You must be able to explain the problem, why it was a problem, and how it was fixed.

Bug #1 — No Database Schema (BLOCKER)

Problem

The migrations/ folder was empty. The entire application assumed the users, vehicles, auctions, and bids tables existed, but no SQL to create them had ever been written.

Effect: The server would start but crash with a PostgreSQL error on the very first API call.

Fix

Created 001_initial_schema.sql with all four tables. Applied it with: `psql -U postgres -d vehicle_auction -f migrations/001_initial_schema.sql`

Subsequent schema changes were made as additional numbered migrations (002 through 008), never by modifying 001.

Bug #2 — Broken Auth Flow: Users Could Never Log In (BLOCKER)

Problem

`authService.js: login()` contained a hard 403 block: `if (!user.is_verified) { throw 403 }`

There was no endpoint to set `is_verified` to true — no email verification, no admin-verify endpoint. Registration worked but login was permanently blocked for all users.

The intent was that unverified users can log in but cannot bid. The implementation accidentally prevented login entirely.

Fix

Removed the 403 block from `login()`. Users can now always log in after registering.

`isVerified` status is embedded in the JWT payload at login time.

A new admin-only endpoint `PATCH /api/users/:id/status` was created for admins to set status to verified or rejected.

`bidService.placeBid()` checks `user.isVerified` from the JWT — unverified buyers get 403 specifically on bid placement, not on login.

After admin verifies a user, the user must log in again to get a new JWT with `isVerified: true`. (Old token still says false until expiry.)

Bug #3 — Anyone Could Self-Register as Admin (SECURITY HOLE)

Problem

`routes/auth.js` Joi schema: `role: Joi.string().valid('admin', 'seller', 'buyer').required()`

A POST `/api/auth/register` with "role": "admin" in the body would create a full admin account. Any malicious user could become an admin.

Fix

Route schema changed to: `role: Joi.string().valid('buyer').required()` — only buyer role is accepted from public registration.

A second guard added in `authService.register()`: `const PUBLIC_ROLES = new Set(['buyer']);` — service-level enforcement so the rule holds even if the route validation is bypassed.

Admins are created only via a one-time seed script (`scripts/seed-admins.js`) that inserts directly into the database. No HTTP endpoint for admin creation exists.

Standard security principle: admin is a trust boundary, not a user preference. Server decides roles, not clients.

Bug #4 — Bids Not Validated Against Auction State (LOGIC GAP)

Problem

`bidService.placeBid()` contained a TODO comment saying "enforce auction status and timing rules" but had no actual auction lookup.

A buyer could place a bid on: an auction that had ended, an auction that had not started yet, a non-existent auction. All bids would be accepted.

Fix

`placeBid()` now fetches the auction by ID and uses `effectiveStatus()` to compute the real status (accounting for timestamp-based transitions).

Status checks: if draft → 400 'Auction has not started yet'. If ended → 400 'Auction has ended'. If not active → 400 'Auction is not active'.

`effectiveStatus()` is a pure function in `auctionService.js` that checks DB status against current timestamps and returns the correct status without a DB write.

Bug #5 — Bids Scoped to Vehicle Not Auction (ARCHITECTURE PROBLEM)

Problem

Original bids table had `vehicle_id` instead of `auction_id`. If a vehicle was re-auctioned, `findHighestBid(vehicleId)` would return a bid from the previous auction as the current high bid. This is a fundamental architectural error — bids belong to an auction event, not a physical vehicle.

Fix

Migration `002_bids_auction_id.sql`: dropped `vehicle_id` from bids, added `auction_id` FK to auctions.

All bid-related queries now scope to `auction_id`. `findHighestBid(auctionId)` returns only bids from the current auction.

Indexes added: `idx_bids_auction_id` and `idx_bids_auction_amount` (for fast highest-bid queries).

Bug #6 — updateVehicle Wiped Unset Fields to NULL (LOGIC GAP)

Problem

vehicleService.updateVehicle() passed the raw request body straight to the repository. The SQL UPDATE set ALL columns.

If a caller sent only { title: "New Title" }, the UPDATE query would set make, model, year, description, images etc. to undefined → NULL in PostgreSQL.

Partial updates were completely broken.

Fix

vehicleService.updateVehicle() now fetches the existing record first, then merges with the payload:

```
const merged = { title: payload.title ?? existing.title, make: payload.make ?? existing.make, ... }
```

The nullish coalescing operator (??) means: use the payload value if provided, otherwise keep the existing database value.

This pattern is called "partial update merge" and is the standard approach for PUT/PATCH operations.

Bug #7 — Auction Status Never Auto-Transitioned (LOGIC GAP)

Problem

Auctions had starts_at and ends_at timestamps and a status column (draft/active/ended), but nothing actually changed the status based on time.

An auction would stay as "draft" forever even after its starts_at time had passed. Buyers would see incorrect statuses.

Fix — Two-Layer Approach

Layer 1 — On-Read Correction: effectiveStatus(auction) is called every time an auction is fetched. If timestamps say status should differ from stored value, it updates the DB row immediately.

Layer 2 — Cron Scheduler: src/scheduler.js runs every minute using node-cron. Runs two UPDATE queries: draft→active when starts_at has passed; active→ended when ends_at has passed.

Why both? The cron proactively updates DB even when no request comes in. The on-read correction is a safety net for the gap between cron runs (up to 59 seconds).

The scheduler only runs in server.js (production) — tests import app.js directly so the cron never starts during testing.

Bug #8 — No Winner Tracking When Auction Ends (LOGIC GAP)

Problem

There was no mechanism to record who won an auction. No winning_bid_id, no winner_user_id on the auction table.

The core business requirement — selecting and announcing a winner — was entirely unimplemented.

Fix

Migration 003_auction_winner.sql: added winning_bid_id FK to auctions.

Two new service methods: `closeAuction()` auto-selects highest bidder; `selectWinner()` allows manual pick.

GET `/api/auctions/:id/winner` public endpoint: returns winner name, email, and winning bid amount.

Frontend: admin sees "Close Auction" button and a bid ladder for manual selection. Winner page displays result publicly.

`findWonByUser(userId)`: returns auctions where the winning bid belongs to that user — used for the buyer profile page.

Bug #9 — No GET /me Endpoint (MISSING FEATURE)

Problem

The JWT contains sub (user ID), role, email, isVerified — but there was no endpoint to fetch the full current user profile.

For the profile page and verification status display, the frontend needed up-to-date user data, not just the (potentially stale) JWT payload.

Fix

Added GET `/api/users/me` — requires `authRequired` middleware, returns current user including `verificationStatus` from live DB.

Also added GET `/api/users` — admin-only, returns all users for the user management panel.

Bug #10 — Auction Routes Had No Input Validation (MISSING)

Problem

`routes/vehicles.js` used `validate(vehicleSchema)` middleware on all mutating endpoints.

`routes/auctions.js` had NO Joi validation at all on POST or PUT — any garbage body went straight to the service.

Fix

`auctionCreateSchema`: `vehicleId` required, optional `title/description/status/startsAt/endsAt/minIncrement/reservePrice`.

`auctionUpdateSchema`: all fields optional (service-level merge handles partial updates).

`validateQuery()` middleware was also added to the vehicles list route for query parameter coercion.

Bug #11 — isVerified Returned as undefined in Register Response

Problem

`authService.register()` returned `isVerified: user.isVerified` but the DB row uses `snake_case: user.is_verified`.

The register response returned `isVerified: undefined` to all clients.

Discovered by the test suite after writing automated tests — this is why tests are valuable.

Fix

Changed to: `isVerified: user.is_verified === true` (or `user.verification_status === 'verified'` after migration 005).

This was a real production bug found and fixed by the test: `expect(res.body.isVerified).toBe(false)`.

Bug #12 — GET /auctions/:id/bids Required Auth (GUEST LOCKOUT)

Problem

The bid list route had `authRequired` middleware, meaning guests and unauthenticated visitors could not see bid history.

The frontend also only fetched bids if (user) — logged-out visitors never saw the highest bid.

Requirement says bid history should be public/transparent.

Fix

Removed `authRequired` from GET /api/auctions/:id/bids — now fully public.

`bidService.listBidsForAuction()` no longer checks or uses the user parameter.

Frontend fetches bids for all visitors regardless of authentication state.

`bidRepository.findByAuction()` was also cleaned up to only select necessary fields (no `user_id` in public response).

6. Key Design Decisions — Deep Dive

6.1 JWT Authentication — How It Works End-to-End

What is a JWT?

A JSON Web Token is a Base64-encoded string in three parts: header.payload.signature. The header specifies the algorithm. The payload contains claims (user data). The signature is a cryptographic hash of header+payload using the server's secret.

How this project uses JWTs

1. User logs in → `authService.login()` creates token:
`jwt.sign({ sub: user.id, role: user.role, email: user.email, isVerified: user.verification_status === 'verified' }, JWT_SECRET, { expiresIn: '1h' })`
2. Client stores token in `localStorage`.
3. Every request: Axios interceptor reads `localStorage` and adds:
`Authorization: Bearer <token>`
4. Protected routes run `authRequired` middleware:
`req.user = jwt.verify(token, config.jwtSecret)`
 → Sets `req.user` to the decoded payload for the rest of the request
5. Services check `req.user.role`, `req.user.isVerified` as needed.

Important Limitation: Stale JWT

The `isVerified` flag in the JWT is set at login time. If an admin verifies a buyer AFTER they log in, their existing token still says `isVerified: false`.

The buyer must log out and log back in to get a new token reflecting their verified status.

For the profile page, the frontend fetches `/api/users/me` to get the live database status — this is always current.

This is a known, acceptable tradeoff for stateless JWTs. The alternative (server-side session store) is more complex.

6.2 Reserve Price Privacy

The `reserve_price` is stored in the database but is never sent to buyers in API responses.

```
// auctionService.js - stripReserve()
function stripReserve(auction, user) {
  const reserveMet = auction.reserve_price == null ? null
    : Number(auction.highest_bid) >= Number(auction.reserve_price);
  if (user?.role === 'admin') return { ...auction, reserve_met: reserveMet };
  const { reserve_price, ...rest } = auction; // destructure to remove it
  return { ...rest, reserve_met: reserveMet }; // send indicator but not amount
}
```

`reserve_met` is always included (as a boolean or null) so buyers can see an indicator on the UI without knowing the actual amount. This is privacy by design — sensitive data is stripped at the service layer, not filtered in the frontend.

6.3 Admin Seed Script vs. Registration

Admins cannot be created through the public registration API. The only way to create an admin is through scripts/seed-admins.js — a Node.js script that runs once during deployment. It reads from .env, connects directly to the database, and inserts the admin accounts.

Why? An "admin" is a trust boundary, not a user preference. The server (specifically a database-level script run by the developer) decides who is admin — never the client. This prevents any attacker from simply posting role: "admin" to the register endpoint.

6.4 Auction Status — The Dual-Layer Approach

Auction status (draft/active/ended) is managed by two complementary mechanisms working together:

```
Layer 1 - effectiveStatus() in auctionService.js:
  Pure function. Called on every auction read.
  draft + starts_at <= now → returns "active"
  active + ends_at <= now → returns "ended"
  Also triggers a DB write-back if status differs.

Layer 2 - Cron scheduler in scheduler.js:
  Runs every minute via node-cron.
  Two bulk UPDATE queries on the whole auctions table.
  Proactively corrects status even when no one is reading.
```

This dual approach means the displayed status is always accurate and the database row reflects reality within at most one minute, even if no requests come in.

6.5 Image Storage — Cloudinary

Vehicle images are not stored in the database or the server filesystem. They are uploaded directly from the browser to Cloudinary (a cloud image storage service) using an unsigned upload preset. The server only stores the resulting Cloudinary URL in the images TEXT[] column.

Why? Storing binary files in PostgreSQL or on the server is inefficient and complex. Cloudinary handles storage, CDN delivery, image transformations, and scaling. The server stays stateless.

How it works: Admin selects images in VehicleFormPage → uploadImage() in cloudinary.js posts directly to Cloudinary API → returns secure_url → URL stored in form state → included in the vehicle create/update API call.

7. Complete API Reference

7.1 Auth Endpoints

Method	Path	Auth	Description
POST	/api/auth/register	None	Register as buyer. Body: { name, email, password, role: "buyer" }
POST	/api/auth/login	None	Login. Returns { token, user: { id, email, role, name, isVerified } }

7.2 User Endpoints

Method	Path	Auth	Description
GET	/api/users	Admin	List all users with verificationStatus
GET	/api/users/me	Auth	Get own profile from live DB (not JWT)
PATCH	/api/users/:id/status	Admin	Set user status. Body: { status: "verified" "rejected" }

7.3 Vehicle Endpoints

Method	Path	Auth	Description
GET	/api/vehicles	None	List vehicles. Query params: status, make, model, yearMin, yearMax, priceMin, priceMax, search, page, limit, sortBy, order
GET	/api/vehicles/:id	None	Single vehicle
POST	/api/vehicles	Admin	Create vehicle with all fields
PUT	/api/vehicles/:id	Admin	Update vehicle (full replace — merged in service layer)
DELETE	/api/vehicles/:id	Admin	Delete vehicle

7.4 Auction Endpoints

Method	Path	Auth	Description
GET	/api/auctions	None	List all auctions with effective status applied
GET	/api/auctions/won/me	Auth	Buyer's won auctions — MUST be before /:id route
GET	/api/auctions/:id	None	Single auction (reserve_price stripped for non-admins)
POST	/api/auctions	Admin	Create auction. vehicleId required.

Method	Path	Auth	Description
PUT	/api/auctions/:id	Admin	Update auction fields
DELETE	/api/auctions/:id	Admin	Delete auction
POST	/api/auctions/:id/close	Admin	Close auction — auto-assigns highest bid as winner. 409 if already ended.
POST	/api/auctions/:id/winner	Admin	Manually select a specific bid as winner. Body: { bidId }
GET	/api/auctions/:id/winner	None	Get winner details — public after admin selects winner
GET	/api/auctions/:id/bids	None	Bid history — public, sorted by amount DESC
POST	/api/auctions/:id/bids	Verified Buyer	Place a bid. Body: { amount }

7.5 JWT Payload

```
{ sub: userId, role: "admin" | "buyer", email: "...", isVerified: true | false }
```

7.6 Error Response Format

```
{ "message": "Human-readable description", "details": ["optional", "array"] }
```

HTTP status codes:

```
400 — Bad input / business rule violation
401 — Missing or invalid token
403 — Authenticated but wrong role, or unverified buyer
404 — Resource not found
409 — Conflict (duplicate email, winner already set, duplicate chassis)
500 — Unexpected server error
```

8. Development Workflow — How This Was Built

8.1 Methodology

An Agile-inspired iterative methodology was used with four sprints, each delivering a vertical slice of working functionality across frontend, backend, and database — not a single tier in isolation. This approach was chosen because:

- Requirements benefit from early UI/UX feedback before full implementation.
- The bidding module carries technical risk that needs early validation.
- Real client feedback can be incorporated after each sprint.

Sprint Breakdown

Sprint	Status	Deliverable
Sprint 1	Complete	End-to-end user registration and login: DB schema, auth API, login UI.
Sprint 2	Complete	Vehicle listings: API endpoints, grid UI, detail page, filtering.
Sprint 3	Complete	Bidding module: auction management, bid placement, countdown timers.
Sprint 4	Complete/Ongoing	Admin dashboard, winner selection, polish, UAT.

8.2 How to Run the Project

Database Setup

```
# 1. PostgreSQL must be installed and running
# Add to PATH on Windows: $env:PATH = "C:\Program Files\PostgreSQL\18\bin;" +
$env:PATH

# 2. Create the database
psql -U postgres -c "CREATE DATABASE vehicle_auction;"

# 3. Run ALL migrations in order
psql -U postgres -d vehicle_auction -f Backend/migrations/001_initial_schema.sql
psql -U postgres -d vehicle_auction -f Backend/migrations/002_bids_auction_id.sql
psql -U postgres -d vehicle_auction -f Backend/migrations/003_auction_winner.sql
psql -U postgres -d vehicle_auction -f Backend/migrations/004_vehicle_fields.sql
psql -U postgres -d vehicle_auction -f
Backend/migrations/005_user_verification_status.sql
psql -U postgres -d vehicle_auction -f
Backend/migrations/006_auction_min_increment.sql
psql -U postgres -d vehicle_auction -f
Backend/migrations/007_auction_reserve_price.sql
psql -U postgres -d vehicle_auction -f
Backend/migrations/008_vehicle_chassis_unique.sql

# 4. Create admin accounts
cd Backend && node scripts/seed-admins.js
# Creates: owner@example.com / ChangeMe1
```

Environment Files

```
# Backend/.env
PORT=3000
DATABASE_URL=postgresql://postgres:21108@localhost:5432/vehicle_auction
JWT_SECRET=your_secret_here
JWT_EXPIRES_IN=1h
BCRYPT_ROUNDS=10
NODE_ENV=development

# Frontend/.env
VITE_CLOUDINARY_CLOUD_NAME=dvko4jtlc
VITE_CLOUDINARY_UPLOAD_PRESET=vehicle_img
```

Starting the Servers

```
# Backend (port 3000)
cd Backend && npm run dev

# Frontend (port 5173, proxies /api to :3000)
cd Frontend && npm run dev

# Verify backend is alive
GET http://localhost:3000/api/health
```

8.3 Testing

Test Setup

```
# Create a separate test database (NEVER run tests against dev DB)
psql -U postgres -c "CREATE DATABASE vehicle_auction_test;"
psql -U postgres -d vehicle_auction_test -f migrations/001_initial_schema.sql
# ... apply all migrations ...

# Create Backend/.env.test
DATABASE_URL=postgres://postgres:21108@localhost:5432/vehicle_auction_test
JWT_SECRET=test-secret
```

Running Tests

```
npm test                # Run all tests
npm run test:report      # Generate test-results/results.json
```

Test Coverage

Test File	What It Tests
auth.test.js	Register buyer, block admin registration, duplicate email, bad password, login + token, 401 cases.
vehicles.test.js	Public list, create/update/delete with 401/403/200/204 per role. Filtering and pagination suite.
users.test.js	Verify/reject flow, 401/403/404/409 cases, fresh token has isVerified: true, GET /me.
bids.test.js	Unverified buyer blocked, verified buyer succeeds, bid outbid rules, min_increment enforcement, admin cannot bid.

Test File	What It Tests
auctions.test.js	Auction CRUD, close auction, manual winner selection, GET /winner, status auto-transitions.

Windows-Specific Jest Issue Found During Development

On Windows, the npm test script running `node_modules/.bin/jest` failed with 'SyntaxError: missing) after argument list' because `.bin/jest` is a bash script.

Fix: Changed the test script to use `node_modules/jest/bin/jest.js` (the actual Node.js entry point) instead of `.bin/jest`.

This is a common Windows gotcha with npm scripts that reference `.bin/` executables.

ESM + Jest Issue

The project uses 'type': 'module' in package.json (ES Modules). Jest does not support ESM by default.

Fix: Run Jest with `--experimental-vm-modules` flag. The test script becomes:

```
node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand --forceExit
```

9. Frontend Deep Dive

9.1 Tailwind CSS v4 — How It Differs

This project uses Tailwind CSS v4, which has a significantly different setup from v3.

Feature	Tailwind v3 (old)	Tailwind v4 (this project)
Configuration	tailwind.config.js file	No config file needed. Done via CSS.
PostCSS setup	postcss.config.js required	No PostCSS config. Uses Vite plugin.
Import in CSS	@tailwind base/components/utilities	@import "tailwindcss"
Vite integration	postcss plugin	@tailwindcss/vite Vite plugin
npx tailwindcss init	Works	Does NOT work — npx tailwindcss init fails

```
// vite.config.js
import tailwindcss from '@tailwindcss/vite'
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: { proxy: { '/api': { target: 'http://localhost:3000', changeOrigin: true } } }
})

// src/index.css
@import "tailwindcss";
@plugin "@tailwindcss/forms";
```

9.2 AuthContext — How It Works

AuthContext is the central auth store for the frontend. It uses React Context API so any component can access auth state without prop drilling.

```
// State initialized from localStorage on page load — no API call needed
const [token, setToken] = useState(() => localStorage.getItem('token'))
const [user, setUser] = useState(() => {
  const t = localStorage.getItem('token')
  return t ? decodeToken(t) : null // atob() decode, no network call
})

// login() called after successful API login
function login(newToken) {
  localStorage.setItem('token', newToken)
  setToken(newToken)
  setUser(decodeToken(newToken))
}

// Context provides: token, user, login, logout, isAdmin, isVerified
// isAdmin = user?.role === 'admin'
// isVerified = user?.isVerified === true
```

Inactivity auto-logout: After 30 minutes of no user interaction (no mousemove, click, keydown, etc.), the session expires. This fires a session-expired custom event which App.jsx listens to and redirects to /login with a message.

9.3 Vite Proxy — How /api Works

The frontend runs on port 5173. The backend runs on port 3000. In development, Vite's proxy forwards any request starting with /api from 5173 to port 3000 automatically. This means:

- No CORS issues in development.
- The Axios baseURL is simply /api — not http://localhost:3000/api.
- In production, you configure Nginx to do the same proxying.

9.4 Route Ordering — A Critical Bug to Know

Route Ordering Problem: /won/me vs /:id

In routes/auctions.js, the route GET /api/auctions/won/me MUST be defined BEFORE GET /api/auctions/:id.

If /:id comes first, Express matches /won/me as id="won" and then tries to GET /auctions/won — which fails.

This is a classic Express routing gotcha: specific static routes must precede dynamic parameter routes.

The code has a comment: // IMPORTANT: /won/me must be BEFORE /:id to avoid route conflict

9.5 Frontend Pages — Key Behaviours

AuctionDetailPage

- Polls auction + bids every 15 seconds while status is active — ensures live bid updates without WebSocket complexity.
- Lightbox gallery: hero click opens fullscreen. Arrow keys / ← → navigate images. Esc closes. Click to zoom 150%.
- Reserve price indicator: reserve_met === null → no reserve. false → reserve not met (red). true → reserve met (green). Admin also sees actual LKR amount.
- Admin controls: Close Auction button + bid ladder for manual winner selection.

VehicleListPage

- Paginated with 12 per page. All filter state is URL-synced (using URLSearchParams). Back/forward browser navigation works correctly.
- Skeleton loading with pulsing grey cards while data fetches.

ProfilePage

- Calls GET /api/users/me on mount — gets live verification status from DB, not stale JWT data.
- Shows won auctions section for buyers.

AdminPage

- Three tabs: Vehicles, Auctions, Users.

- After verifying/rejecting a user, dispatches a custom browser event user-status-changed. The Navbar listens for this event and re-fetches the pending user count badge immediately, without a page reload.

10. End-to-End: How a Sale Happens

Understanding this complete flow is essential for the viva. Every step maps to specific code.

Step	Who Does It	What Happens (Code)
1	Admin	Creates Vehicle (status: listed) via POST /api/vehicles. Images uploaded to Cloudinary first.
2	Admin	Creates Auction linked to vehicle, sets starts_at, ends_at, optional reserve_price via POST /api/auctions.
3	System (cron)	Scheduler runs every minute. When starts_at passes: UPDATE auctions SET status='active'.
4	Buyer	Registers via POST /api/auth/register. Account status: pending.
5	Admin	Verifies buyer via PATCH /api/users/:id/status { status: "verified" }.
6	Buyer	Logs in again to get new JWT with isVerified: true.
7	Buyer	Places bids via POST /api/auctions/:id/bids. Each bid must be >= highest + min_increment.
8	System (cron)	When ends_at passes: UPDATE auctions SET status='ended'.
9	Admin	Either: POST /api/auctions/:id/close (auto-picks highest bidder), or POST /api/auctions/:id/winner { bidId } (manual pick).
10	System	GET /api/auctions/:id/winner becomes available publicly. Buyer sees "You won!" banner on next login.
11	Buyer	Sees won auction in Profile page. Transaction proceeds offline.

11. Code Change Challenge — Viva Preparation Guide

The viva panel will change some code and ask you to perform a task. This chapter gives you the mental model to handle any change they make.

11.1 How to Read Any File Quickly

For each file, ask these three questions in under 30 seconds:

1. What layer is this? (Route/Controller/Service/Repository/Component)
2. What resource does it operate on? (vehicles/auctions/bids/users)
3. What are the exports? (what functions does it expose?)

11.2 Common Task Types and Where to Look

If they ask you to...	Look in...
Change a business rule (e.g., bid must be 10% higher)	src/services/bidService.js — placeBid()
Add a new field to an entity	Migration SQL + repository + service + route Joi schema + frontend form
Change who can access an endpoint	src/routes/*.js — the requireRole([...]) middleware call
Change what an endpoint returns	src/services/*.js — the service method shapes the response
Fix a database query	src/repositories/*.js
Add a new API endpoint	Route → Controller → Service → Repository (in that order)
Change auction status logic	src/services/auctionService.js — effectiveStatus() and withEffectiveStatus()
Change frontend display logic	src/pages/*.jsx — find the relevant page component
Change what the Navbar shows	src/components/Navbar.jsx
Change redirect after login	src/pages/LoginPage.jsx — after login() call
Add a filter to vehicle listing	vehicleRepository.findAll() conditions array + routes/vehicles.js listQuerySchema

11.3 Tracing a Request End-to-End

Practice this mental trace for any feature. Example: buyer places a bid.

```
POST /api/auctions/:auctionId/bids
```

1. src/routes/auctions.js
 → router.use('/:auctionId/bids', bidRoutes)
 → mergeParams: true allows bidRoutes to see :auctionId
2. src/routes/bids.js

```

→ authRequired middleware: verifies JWT, sets req.user
→ requireRole(['buyer']): admins cannot bid
→ validate(bidSchema): Joi checks body.amount is positive number
→ placeBidController

3. src/controllers/bidController.js
→ calls bidService.placeBid(req.params.auctionId, req.body.amount, req.user)

4. src/services/bidService.js - placeBid()
→ if (!user.isVerified) → 403
→ auctionRepository.findById(auctionId) → get auction
→ effectiveStatus(auction) → must be "active"
→ bidRepository.findHighestBid(auctionId)
→ check amount >= highest + min_increment
→ bidRepository.create({ auctionId, userId: user.sub, amount })

5. src/repositories/bidRepository.js
→ INSERT INTO bids (auction_id, user_id, amount) VALUES ...
→ returns new bid row

6. Response: 201 with bid object

```

11.4 Things That Are Easy to Forget

Common Mistakes Under Pressure

1. Adding a column: must update the repository INSERT/UPDATE AND the Joi validation schema in routes AND the service merge logic.
2. Changing a route: the route file determines WHO can access it (middleware). The service file determines WHAT they can do.
3. Frontend API calls: never add Axios calls directly in a React component. Always add to src/api/*.js and import from there.
4. The /won/me route must stay BEFORE /:id in auctions routes.
5. After schema changes, apply the migration to BOTH dev database AND test database.
6. The bids route uses mergeParams: true — this is what allows it to see the :auctionId parameter from the parent auctions router.
7. reserve_price is always stripped from non-admin responses in auctionService.js — if you add a new sensitive field, strip it there too.

11.5 If Something Breaks — Debugging Checklist

4. Check the terminal running the backend — the error message is almost always there.
5. Is it a 401? → JWT is missing, expired, or malformed. Check Authorization header.
6. Is it a 403? → User has wrong role OR buyer is unverified. Check req.user.role and req.user.isVerified.
7. Is it a 400? → Joi validation failed. Check the request body against the schema in the route file.
8. Is it a 404? → Resource not found. Check the ID is valid and the resource exists in DB.
9. Did a migration get missed? → Run psql and check \d tablename to see current columns.
10. Frontend not reflecting changes? → Hard refresh (Ctrl+Shift+R). Check Network tab for API response.
11. Bid not showing? → Check if GET /api/auctions/:id/bids requires auth — it should not.

12. Technology Selection

Component	Choice	Why (vs alternatives)
Frontend	React 19 + Vite	Component-based SPA well-suited to dynamic auction UI. Large ecosystem. Vue.js rejected: React has wider adoption for long-term maintainability.
Backend	Node.js + Express	Non-blocking I/O appropriate for concurrent bid submissions. JS across frontend and backend reduces context-switching. Django rejected: Python adds deployment complexity.
Database	PostgreSQL	ACID-compliant transactions essential for bid integrity. Rich support for arrays (images), NUMERIC type for money, UNIQUE constraints. MySQL rejected: weaker SERIALIZABLE isolation and no native array type.
Auth	JWT (jsonwebtoken)	Stateless — no server-side session store required. Claims-based roles simplify middleware. Session cookies rejected: require server-side storage, complicate horizontal scaling.
Styling	Tailwind CSS v4	Utility-first. No custom CSS files. Responsive by default. @tailwindcss/vite plugin eliminates PostCSS config.
Images	Cloudinary	Unsigned upload from browser. CDN delivery. Server stays stateless — no binary file handling needed.
Validation	Joi	Declarative schema validation. stripUnknown removes unknown fields. abortEarly: false returns all errors at once.
Cron	node-cron	Lightweight in-process scheduler. Simple cron syntax. No external queue service needed at this scale.

12.1 Why PostgreSQL over MySQL for This Project Specifically

- NUMERIC(12,2): Exact decimal arithmetic. MySQL also has NUMERIC but PostgreSQL implementation is more mature.
- TEXT[] arrays: Native array support for vehicle images. MySQL has no native array type.
- SERIALIZABLE isolation: Critical for concurrent bid handling (race condition prevention). MySQL's implementation is weaker.
- NULL in UNIQUE: PostgreSQL treats each NULL as distinct in a UNIQUE constraint — multiple vehicles can have NULL chassis_number without conflict.

13. Glossary — Terms You Must Know

Term	Definition
JWT (JSON Web Token)	A signed, Base64-encoded token carrying user claims. Three parts: header.payload.signature. Stateless — server does not store sessions.
bcrypt	Password hashing algorithm with a configurable cost factor. A "salt" is mixed in automatically. Never store passwords in plain text.
Middleware	A function in Express that runs between request and handler. Can modify req/res, call next() to continue, or end the request.
Repository Pattern	A layer that abstracts all database access. Only repositories contain SQL. Services never write SQL.
Migration	An incremental SQL file that describes a schema change. Applied in order. Never modified after being run.
RBAC (Role-Based Access Control)	Access control model where permissions are assigned to roles (admin, buyer) not individuals. Standard security pattern.
Correlated Subquery	A subquery that references columns from the outer query. Used for highest_bid: <code>SELECT MAX(amount) FROM bids WHERE auction_id = a.id.</code>
mergeParams	Express Router option. Allows a child router to access URL parameters from the parent router. Required for <code>/auctions/:auctionId/bids</code> .
Effective Status	The computed real status of an auction based on timestamps, which may differ from the stored DB status if the cron has not run yet.
SPA (Single Page Application)	A web app that loads once and updates the page dynamically via JavaScript, without full page reloads.
Joi	A JavaScript schema validation library. Defines the expected shape of request bodies and query parameters.
ACID	Atomicity, Consistency, Isolation, Durability. Properties that guarantee database transaction reliability.
Vite Proxy	A dev-server feature that forwards requests to another server. <code>/api</code> on port 5173 is proxied to port 3000.
Nullish Coalescing (??)	Returns right-hand value only if left is null or undefined. Used in service merge logic: <code>payload.field ?? existing.field.</code>
pg.Pool	PostgreSQL connection pool. Reuses connections instead of opening a new one per query. Critical for performance under load.
TIMESTAMPTZ	PostgreSQL timezone-aware timestamp. Always stores UTC. Essential for global applications with users in multiple timezones.

— End of Document —